

CS149 Assignment 2: Writeup

1. How I Built the Task Systems

Threads

I made three versions, each one a bit smarter than the last.

The first one, **ParallelSpawn**, is straightforward. Every time someone calls `run()`, I create new threads using a static worker function called `spawnWorker`, give them work, and wait for them all to finish. It works fine but if `run()` gets called hundreds of times all that thread creation adds up and slows things down.

The second one, **ThreadPoolSpinning**, keeps threads alive the whole time instead of creating new ones every call. The threads run in a loop forever using a worker function called `spinWorker`. When there is nothing to do they just keep checking in a loop. This is called spinning. It avoids the cost of creating threads but wastes CPU power while waiting.

The third one, **ThreadPoolSleeping**, is the best of the three. It also keeps threads alive the whole time using a worker function called `sleepWorker`, but instead of spinning, threads go to sleep using condition variables when there is nothing to do. They only wake up when new work arrives. This saves CPU and lets the actual workers run without competition.

Giving Work to Threads

All three versions use dynamic assignment. There is a shared counter protected by a mutex that tracks which task to do next. A thread locks the mutex, reads the counter, takes that task ID, increments the counter, unlocks, then runs the task. This way threads never do the same task twice and no thread sits idle while another is overloaded.

Knowing When Everything is Done

For `ParallelSpawn`, calling `join()` on all threads is enough. When they all finish, all tasks are done.

For `Spinning`, the main thread keeps checking `Dtask >= Ttask` under the mutex until all tasks are marked complete.

For `Sleeping`, the last thread to finish a batch notifies `cv_fnsh` which wakes up the main thread waiting in `run()`.

2. Running Tasks with Dependencies

Tracking What Depends on What

Every batch of tasks gets stored in a struct called Group inside a vector called RGgroup. Each group keeps track of its runnable, total tasks, completed tasks, dependency list, and a counter called wait_left that tracks how many dependencies are still running.

When runAsyncWithDeps() is called, a new group is created, and each dependency is checked to see if it is already finished. The wait_left counter is set to the number of unfinished dependencies. If wait_left is zero the group gets dispatched to the work queue immediately through the dispatch() function.

When a group finishes, the worker goes through all other waiting groups and checks if any of them listed the finished group as a dependency. For each match wait_left is reduced by one. If any group hits zero it gets dispatched immediately.

The Work Queue

Tasks sit in a queue called RDqueue. Each item has a group ID and a task index. Worker threads grab items from the front of the queue, copy the needed data, release the lock, run the task, then re-lock to update the completion counter.

sync()

sync() puts the calling thread to sleep on sync_cv until active reaches zero meaning every registered group has finished. After waking up it clears RGgroup so the next round of calls starts with a clean slate.

run() in Part B

run() just calls runAsyncWithDeps() with an empty dependency vector and then immediately calls sync(). This makes it behave the same as Part A where run() does not return until all tasks are done.

3. What I Noticed About Performance

When Serial Was Just as Fast

On `super_super_light` and `super_light`, serial kept up with or beat some parallel versions. These tests launch 400 batches, but each task barely does any work. At that point the cost of locking, unlocking, and waking threads is more expensive than just running the tasks one by one. Serial has none of that overhead, so it wins on these tiny workloads.

ParallelSpawn vs Thread Pool

On the light tests, `ParallelSpawn` is noticeably slow because it creates and destroys threads on every one of the 400 calls. `Thread Pool Sleeping` handles these much better since the threads are already alive and just need to be pointed at new work.

On heavier tests like `mandelbrot_chunked` and `recursive_fibonacci` the difference shrinks a lot. When tasks take real time to finish, thread creation is a small cost compared to the actual computation.

Why Spinning Sometimes Lost

On my 4-core machine, spinning threads eat up all available CPU even when waiting. This means they compete with threads actually doing work. On the 16-core grading machine this is not a problem since there are extra cores to spare.

Part B Results

The async tests showed solid speedups for `Thread Pool Sleeping`. Tests that have many independent groups like `reduction_tree_async` and `fan_in_async` ran noticeably faster than serial because multiple groups execute at the same time in parallel.

All my numbers are about 1.6 to 2 times slower than reference. This is simply because the reference was measured on a 16 core AWS machine and I only have 4 cores locally. The results on the grading machine will be much closer to the reference.